

3SE2 – Übung 4

Themen: Backend mit Service-Layer, Branches, Merging, Aggregations

Ab dieser Woche darfst du die Übung alternativ auch in deinem Einzelprojekt umsetzen und dies anstelle der Übung abgeben.

Folgendes wird erwartet.

- Das Projekt enthält einen Sprint mit mehreren Issues für die aktuelle Woche.
- Das Projekt enthält einen Endpoint bei dem ein Zustand geprüft wird.
- Das Projekt enthält einen Endpoint mit einer Aggregation
- Das Projekt verwendet einen Service sinnvoll
- Das Projekt enthält mehrere Branches, die gemerged wurden.

Issues erstellen

Erstelle die unten beschriebenen Issues und plane sie für Sprint 4. Beschreibe jeweils die Anforderung und wie diese getestet werden sollen.

9	🟢 Fix company validation for new jobs #9	backend bug	Ready	Iteration 4
10	🟢 API endpoint to assign jobs #10	backend enhancement	Ready	Iteration 4
11	🟢 API endpoint to complete jobs #11	backend enhancement	Ready	Iteration 4
12	🟢 API endpoint for job dashboard #12	backend enhancement	Ready	Iteration 4

Prüfen der Company bei neuen Jobs (Bug)

Aktuell ist es möglich einen Job mit ungültiger companyId zu erstellen. Der POST-Endpoint /api/job soll wie folgt ergänzt werden:

POST	/api/job	Fügt einen neuen Job hinzu
Request Body	<pre>{ "description": "Review Order Class", "jobType": "REVIEW", "earnings": 150.0, "companyId": "12" }</pre>	
Responses	CREATED	
	BAD_REQUEST	Falls die companyId ungültig ist
	INTERNAL_SERVER_ERROR	Speichern fehlgeschlagen

Job übernehmen (Feature)

Mit diesem Endpoint soll der Freelancer einen Job übernehmen können.

PUT	/api/service/assignjob	Ordnet einen Job einem Freelancer zu
Request Body	<pre>{ "jobId": "6344ef34efc7c92e1ad83be1", "freelancerId": "ada@gmail.com" }</pre>	
Responses	OK	{ "id": "6344ef34efc7c92e1ad83be1",

		... "jobState": "ASSIGNED", "companyId": "12", "freelancerId": "ada@gmail.com" }
	BAD_REQUEST	Falls kein Job mit dieser ID existiert. Falls der Job nicht mit State NEW ist.

Job abschliessen (Feature)

Mit diesem Endpoint soll der Freelancer einen Job als abgeschlossen markieren können.

PUT	/api/service/completejob	Markiert einen Job als abgeschlossen
Request Body	{ "jobId": "6344ef34efc7c92e1ad83be1", "freelancerId": "ada@gmail.com" }	
Responses	OK	{ ... "jobState": "DONE", "companyId": "12" "freelancerId": "ada@gmail.com" }
	BAD_REQUEST	Falls kein Job mit dieser ID existiert Falls der Job nicht mit State ASSIGNED ist Falls der Job nicht dem im Request Body angegebenen Freelancer zugeordnet ist.

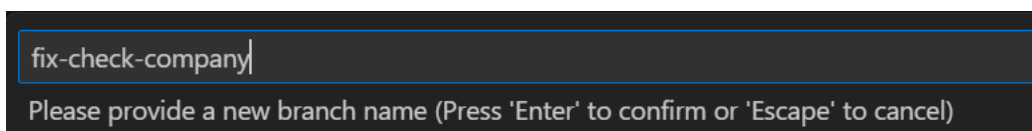
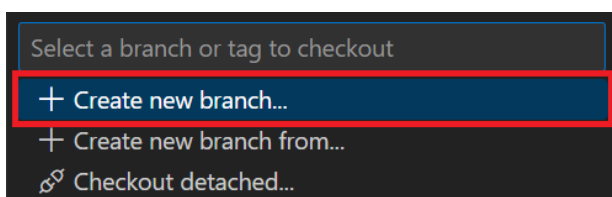
Dashboard für Jobs (Feature)

GET	/api/service/jobdashboard	gibt alle Jobs der gewählten Firma gruppiert nach Zustand zurück.	
Request Parameters	company	Parameter für companyId	
Responses	OK	[{"id": "NEW", "jobIds": ["6344ef34efc7c92e", "..."], "count": "2"}, {"id": "ASSIGNED", "jobIds": ["634a7abac603a20", "..."], "count": "4"}, {"id": "DONE", "jobIds": ["634a8285f7a1a13e", "..."], "count": "1"}]	
Beispiele	/api/service/jobdashboard?company=12	OK, Array mit den Daten im oben gezeigten Format	
	/api/service/jobdashboard?company=123	BAD_REQUEST bei ungültiger CompanyID	
Bemerkung	Existieren keine Jobs eines bestimmten Zustands, fehlt das entsprechende Element.		

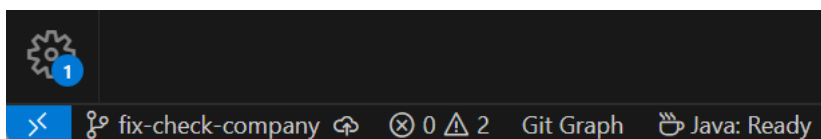
Prüfen der Company bei neuen Jobs

Schiebe das zugehörige Issue in die Spalte «In progress».

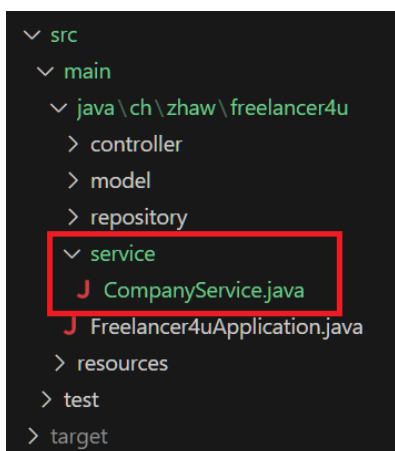
In dieser Übung wollen wir das Arbeiten mit Branches üben. Erstelle deshalb für dieses Issue einen Branch mit dem Namen «fix-check-company». Klicke im Footer auf main (oder master) und erstelle den Branch wie gezeigt.



Eingabe mit Enter abschliessen. Nun sollte im Footer der Branch-Name angezeigt werden.



Es soll neu nicht mehr möglich sein, einen Job mit ungültiger Company ID zu erstellen. Dazu benötigt der JobController auch Informationen aus dem Company Repository. Wir wollen aus den Controllern jedoch nur auf das zugehörige Repo zugreifen und andere Zugriffe an Services delegieren. Erstelle für Company deshalb einen Service in einem separaten Ordner:



Ersetzet den Inhalt der Klasse mit dem folgenden Code:

```
package ch.zhaw.freelancer4u.service;

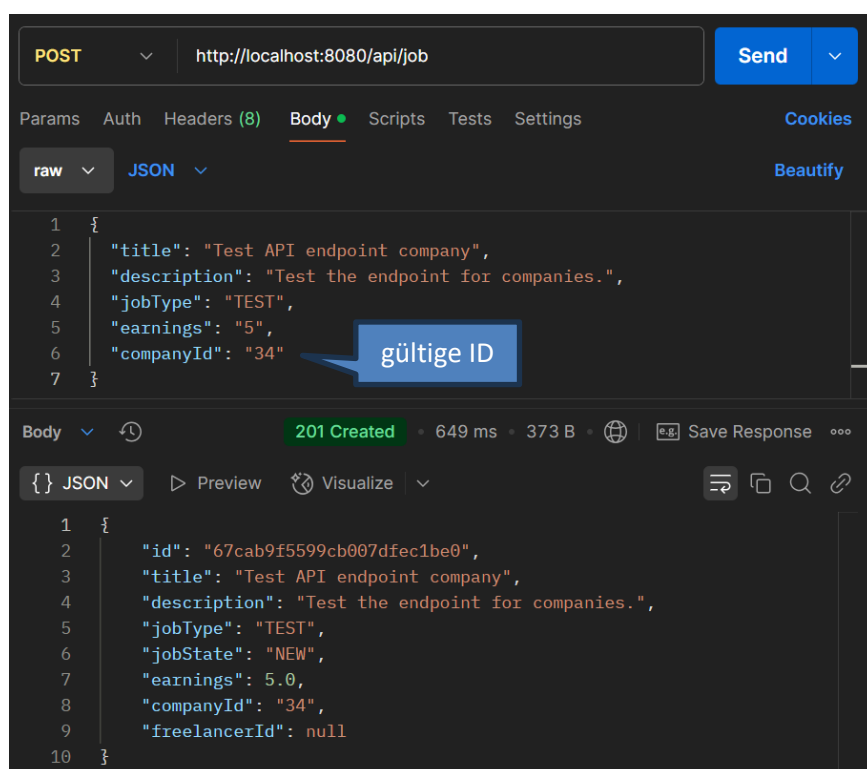
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ch.zhaw.freelancer4u.repository.CompanyRepository;

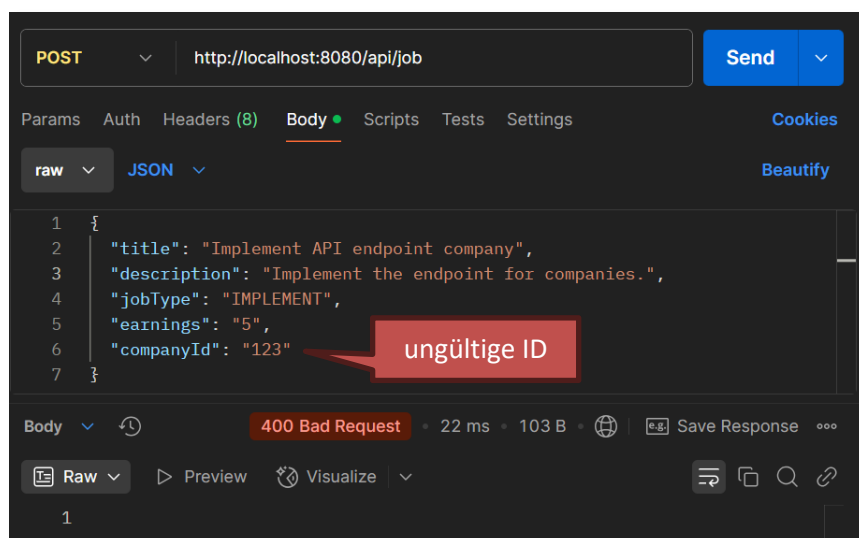
@Service
public class CompanyService {
    @Autowired
    CompanyRepository companyRepository;

    public boolean companyExists(String companyId) {
        return companyRepository.existsById(companyId);
    }
}
```

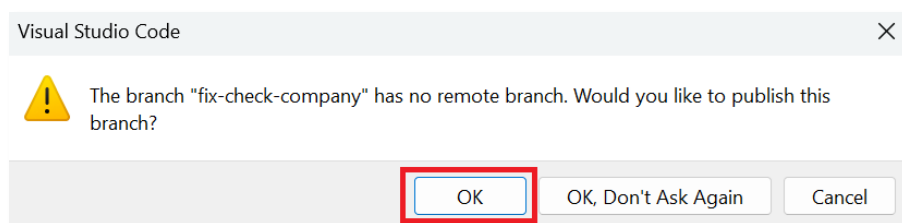
Erklärungen: Die Annotation `@Service` kennzeichnet die Klasse als Service. Der Service muss auf das Company-Repository zugreifen können, deshalb ist dieses mit `@Autowired` verknüpft. Die Methode `companyExists` erhält eine CompanyID als Input. Sie gibt `true` zurück, falls eine Company mit der angegebenen ID existiert, sonst `false`. Die Query `«existsById»` gehört zu den Methoden, die direkt von `MongoRepository` zur Verfügung gestellt werden. Es ist daher keine Ergänzung des Company-Repositories nötig.

Ergänze nun den JobController: Binde den `CompanyService` per `@Autowired` ein und verwende die `companyExists` Methode im POST `/api/job` Endpoint. Passe den Statuscode der Response im Fehlerfall an. Teste die Änderungen mit Postman.

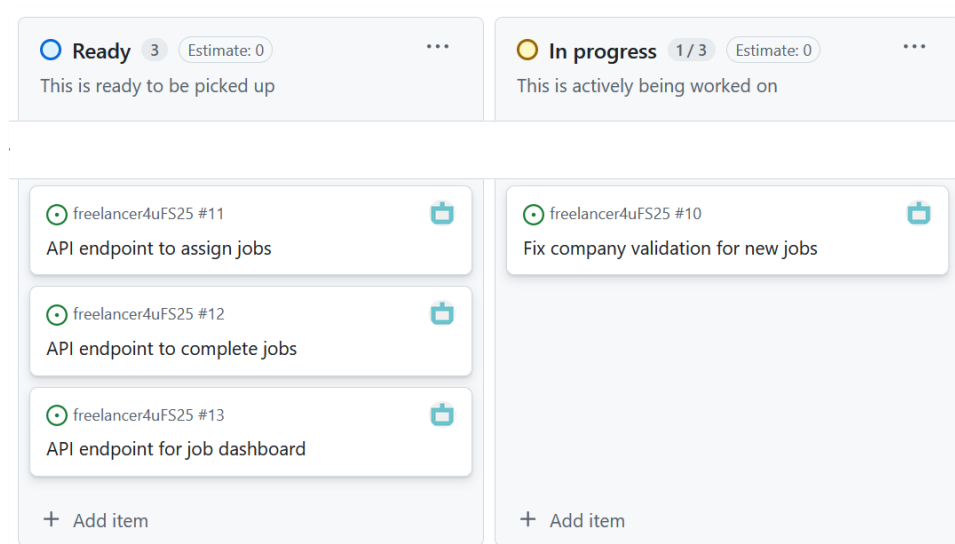




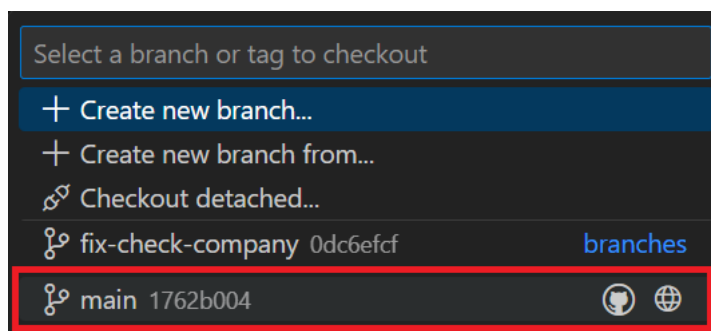
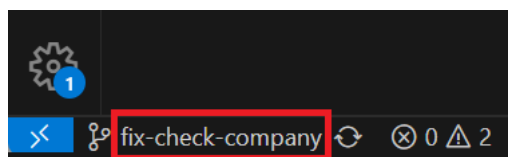
Falls alles funktioniert, committe und pushe deine Änderungen. Schliesse dabei das Issue mit einer entsprechenden Commit-Message ab. Beim Pushen musst du bestätigen, dass der Branch auch im Remote-Repo angelegt wird.



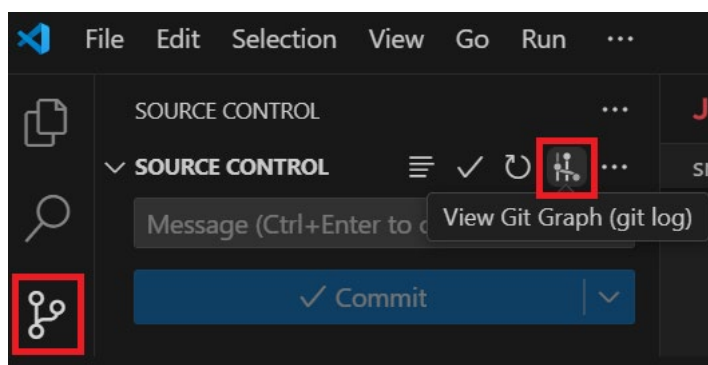
Wenn du dir jetzt das SCURM-Board anschaust, siehst du, dass das zugehörige Issue trotz der entsprechenden Commit-Message immer noch im Zustand «In progress» ist.



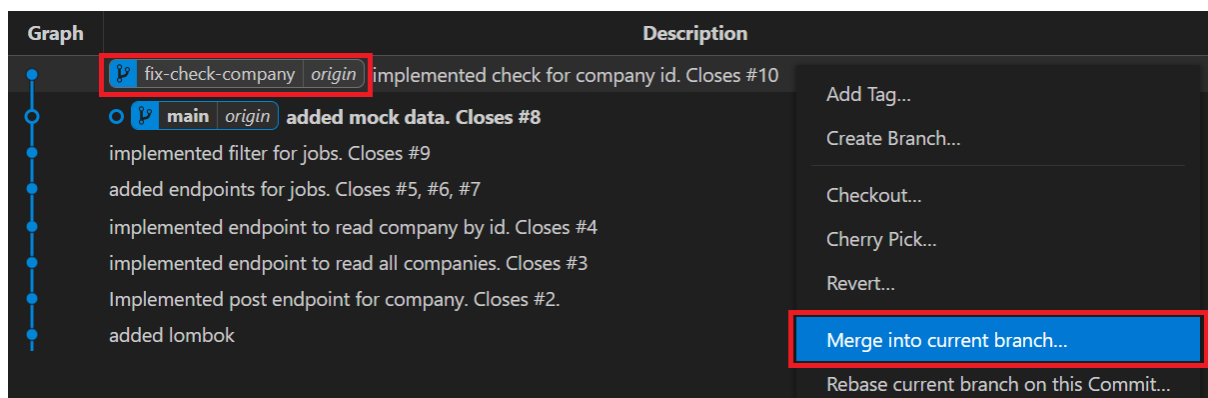
Der Grund dafür ist, dass die Änderungen noch nicht im main (master) Branch angelangt sind. Wechsle im Projekt zunächst zurück zum main Branch:



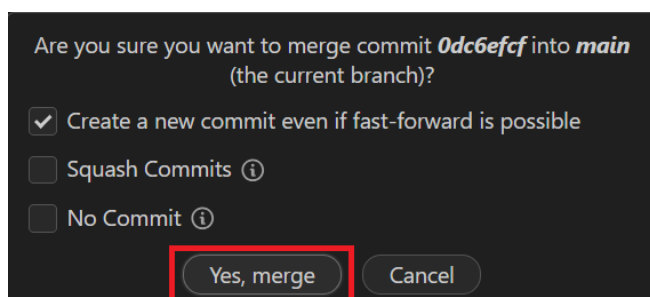
Nun sollte auch im Footer wieder main angezeigt werden. Schau dir den JobController an. Du kannst festzustellen, dass die zuvor gemachten Änderungen verschwunden sind. Die Änderungen aus dem fix-check-company branch, sollen nun in den main Branch gemerged werden. Öffne dazu die Git Graph extension.



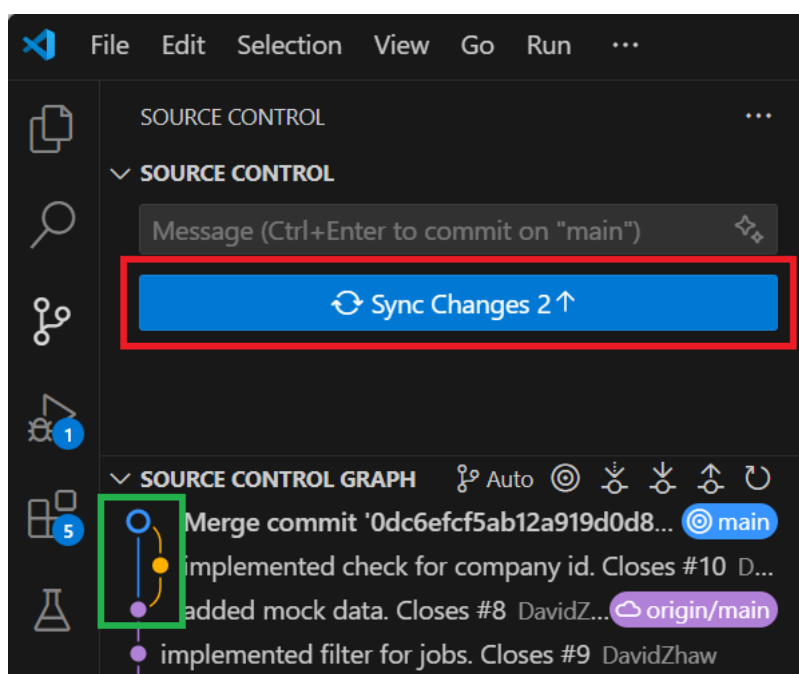
Mache einen Rechts-Click auf den (letzten) Commit im fix-check-company Branch und wähle «Merge».



Du musst du den Merge bestätigen.



Schau dir nochmal den JobController an. Nun sind die Änderungen auch im main Branch drin. Dass gemerged wurde, ist auch im Source Control Graph (in der Abbildung unten grün) sichtbar. Die Änderungen sind aktuell immer noch nur in deinem Lokalen Repo. Bringe die Änderungen nun per Sync in das Remote-Repo.



Schau dir nochmals das SCRUM-Board an. Nun befindet sich das Issue in der Spalte «Done».

Mission one accomplished 😊.

Job übernehmen

Wir machen weiter mit dem Issue «Job übernehmen». Schiebe das Issue in die Spalte «In progress».

Branches können nicht nur in VS Code, sondern auch direkt beim Issue erzeugt werden. Öffne auf GitHub das Issue und erstelle den Branch wie unten gezeigt.

API endpoint to assign jobs #11

Open DavidZhaw/freelancer4u Private

DavidZhaw opened 3 hours ago

DavidZhaw added enhancement 3 hours ago

DavidZhaw self-assigned this 3 hours ago

DavidZhaw added this to Freelancer4u SCRUM Planning 3 hours ago

DavidZhaw moved this to Ready in Freelancer4u SCRUM Planning 3 hours ago

DavidZhaw moved this from Ready to In progress in Freelancer4u SCRUM Planning 1 minute ago

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Assignees: DavidZhaw

Labels: enhancement

Projects: Freelancer4u SCRUM Planning

Status: In progress

Priority: Choose an option

Size: Choose an option

Estimate: Enter number...

Iteration: Iteration 4 • Mar 11 - Mar 17

Milestone: No milestone

Relationships: None yet

Development: Create a branch or this issue or link a pull request.

Create a branch for this issue

Branch name

11-api-endpoint-to-assign-jobs

Repository destination: DavidZhaw/freelancer4u

Branch source: main

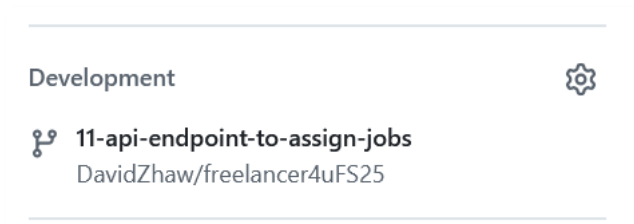
What's next?

☒ Checkout locally

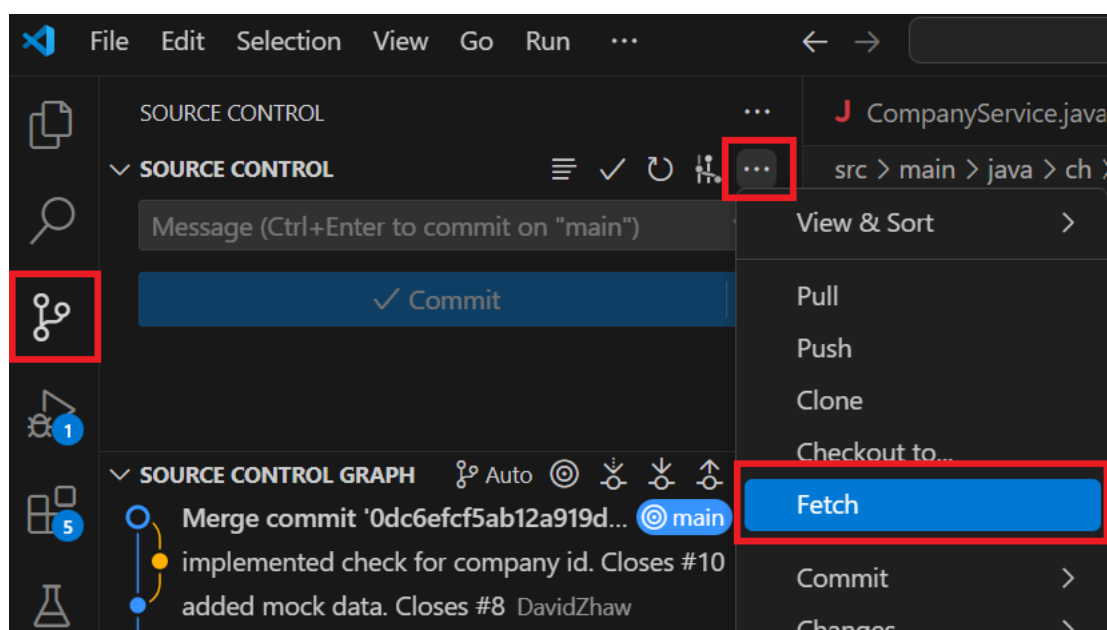
☐ Open branch with GitHub Desktop

Create branch

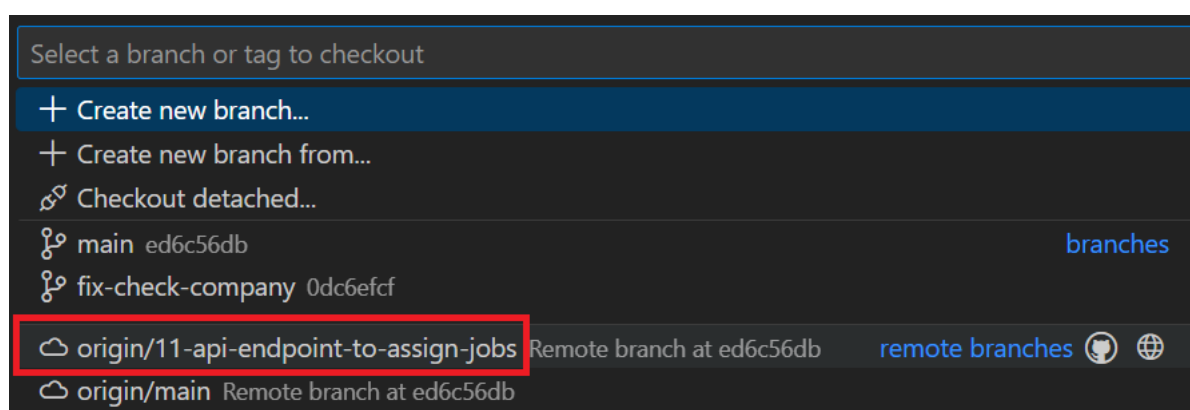
Nun erscheint der Branch auch beim Issue.



Der Branch befindet sich bisher nur im Remote-Repo. Um diesen in dein lokales Repo zu bringen, musst du einen Fetch ausführen.



Nun kannst du nach dem Klick auf den aktuellen Branch (im Footer) den neuen Branch auswählen.



Im Footer soll nun der neue Branch angezeigt werden.

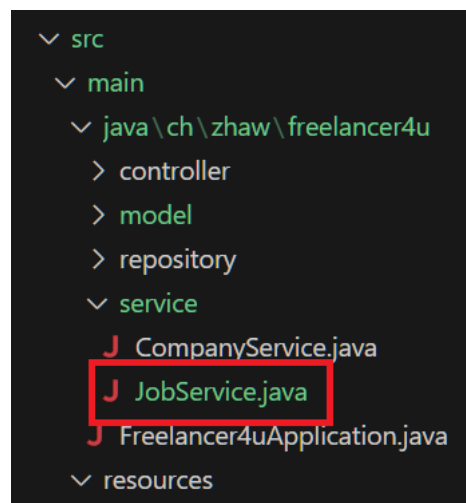
Es gibt zwei Endpoints, welche den Zustand eines Jobs ändern: einen, um Jobs einem Freelancer zuzuteilen und einen, um Jobs abzuschliessen. Beide Endpoints erwarten PUT-Request mit dem gleichen Body-Format:

```
{
  "jobId": "6344ef34efc7c92e1ad83be1",
  "freelancerId": "ada@gmail.com"
}
```

Als erstes erstellen wir deshalb eine passende Datenklasse im Ordner «model». Da diese Datenklasse nicht in die DB geschrieben wird, ergänzen wir den Klassennamen mit DTO. Implementiere die Klasse gemäss folgendem UML-Diagramm und verwende Lombok für die Getter-Methoden und den Konstruktor.

JobStateChangeDTO
- jobId: String - freelancerId: String
+ JobStateChangeDTO() + getJobId(): String + getFreelancerId(): String

Beim «Job übernehmen» handelt es sich nicht um einen einfachen Lese- oder Schreibzugriff auf die gespeicherten Daten. Hier müssen die Anforderungen des zugehörigen Geschäftsprozesses berücksichtigt werden. Die Zustandsübergänge dürfen nur unter bestimmten Bedingungen stattfinden. Wir fügen für diesen Use-Case deshalb ein Service-Layer ein, welches die Anforderungen des Geschäftsprozesses umsetzt. Füge dem Projekt im Ordner «service» die Klasse «JobService» hinzu.



Starte mit dem folgenden Inhalt für die Klasse JobService:

```
package ch.zhaw.freelancer4u.service;

import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ch.zhaw.freelancer4u.model.Job;
import ch.zhaw.freelancer4u.repository.JobRepository;

@Service
public class JobService {
    @Autowired
    JobRepository jobRepository;

    public Optional<Job> assignJob(String jobId, String freelancerId) {
        return Optional.empty();
    }
}
```

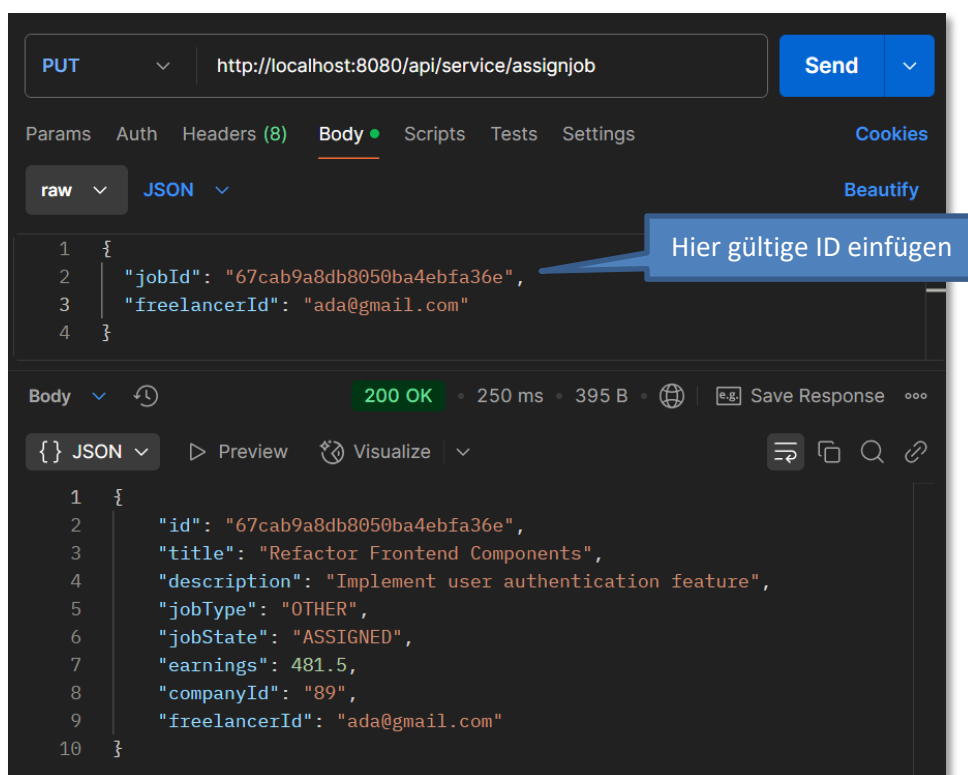
Die Methode assignJob erhält die jobId und das Freelancer-E-Mail als Input. Der Rückgabewert ist ein Optional vom Typ Job. Falls die Zuweisung erfolgreich war, soll das modifizierte Job-Objekt zurückgegeben werden, andernfalls ein leeres Optional.

Implementiere nun die folgenden Anforderungen an die assignJob-Methode:

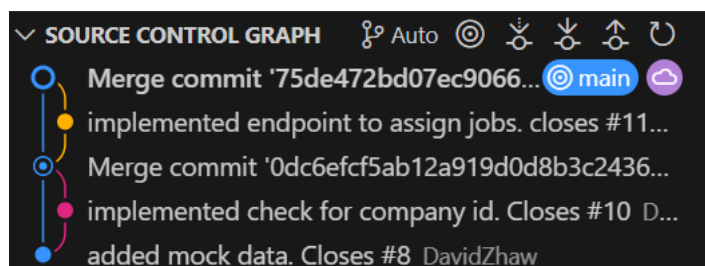
1. Prüfen, ob die jobId gültig ist: Job aus DB lesen und prüfen, ob er zurückgegeben wurde.
2. Prüfen, ob sich der angegebene Job im Zustand NEW befindet.
3. Die Zuweisung vornehmen: Job in den Zustand ASSIGNED setzen und das E-Mail des Freelancers unter FreelancerID eintragen.
4. Die Änderungen in der DB speichern.
5. Neuen Job als Optional zurückgeben. Falls etwas nicht gepasst hat, ein leeres Optional zurückgeben.

Nun fehlt noch der zugehörige Endpoint. Für die Endpoints mit Service-Charakter erstellen wir einen separaten Controller. Füge dem Projekt die Klasse «JobServiceController» im Ordner «controller» hinzu. Hier der Code für den Endpoint. Ergänze den Rest der Klasse selbständig.

```
@PostMapping("/assignjob")
public ResponseEntity<Job> assignJob(@RequestBody JobStateChangeDTO changeS) {
    String freelancerId = changeS.getFreelancerId();
    String jobId = changeS.getJobId();
    Optional<Job> job = jobService.assignJob(jobId, freelancerId);
    if (job.isPresent()) {
        return new ResponseEntity<>(job.get(), HttpStatus.OK);
    }
    return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
}
```



Wenn alles funktioniert, committe und pushe deine Änderungen und schliesse das Issue mit einer entsprechenden Message. Merge anschliessend die Änderungen in den main Branch. Orientiere dich an den Schritten vom letzten Issue. Der Source Control Graph sollte etwa so aussehen:



Prüfe, ob danach das Issue im Zustand «Done» ist. Manchmal muss die Seite neu geladen werden..

Job abschliessen

Ähnlich wie beim Übernehmen müssen beim Abschliessen eines Jobs die Anforderungen des Prozesses umgesetzt werden. Der Zustandsübergang soll deshalb wieder im Service umgesetzt werden. Hier die wichtigsten Punkte:

- Ergänze JobService um die Methode completJob analog zu assignJob
- Ergänze JobServiceController um den Endpoint /completejob

Beachte, dass es für den Service eine zusätzliche Bedingung gibt: ein Job kann nur dann in den Zustand «DONE» überführt werden, wenn der Job demjenigen Freelancer zugeordnet ist, dessen E-Mail im Body des Request unter freelancerId angegeben ist.

Teste mit Postman wiederum alle möglichen Fälle und Ausnahmen. Wenn alles funktioniert, committe deine Änderungen und schliesse mit einer entsprechenden Message das Issue ab.

Branching für diese Teilaufgabe sind freiwillig. 😊

Dashboard für Jobs

Für den Dashboard-Endpoint müssen Daten von mehreren Job-Dokumenten aggregiert werden. Mit Keyword-basierten Query Methode können keine passenden Queries formuliert werden. Es gibt zwei Möglichkeiten für die Umsetzung:

- Alle Jobs von der DB lesen und dann in der Applikation aggregieren. Zum Aggregieren könnte man einen Java-Stream verwenden. Der Nachteil ist, dass dadurch der Webserver stark belastet würde und grosse Datenmengen zwischen der DB und dem Webserver ausgetauscht werden müssten.
- Eine Query an die DB schicken, welche die Daten auf dem Datenbankserver aggregiert. Mit den Aggregation-Pipelines, welche wir in 3DM kennengelernt haben, ist dies möglich. Zudem ist das Aggregieren auf dem DB-Server viel performanter als das Aggregieren in der Applikation.

Wir setzen die Variante mit der Aggregation-Pipeline um. Zunächst entwerfen und testen wir in Compass die passende Pipeline. Wir starten mit einer vereinfachten Variante bei der in der Antwort nur die Anzahl Jobs enthalten sind.

```
[
  {
    $match: {
      companyId: "31"
    }
  },
  {
    $group: {
      _id: "$jobState",
      count: {
        $count: {}
      }
    }
  }
]
```

The screenshot shows the MongoDB Compass interface. The top navigation bar includes 'Documents' (502), 'Aggregations' (selected), 'Schema', 'Indexes' (1), and 'Validation'. Below the navigation bar, there are tabs for '\$match' and '\$group', and buttons for 'Explain', 'Export', 'Run', and 'Options'. The main area displays the aggregation pipeline in a code editor, which is highlighted with a red box. The pipeline is:

```
[
  {
    $match: {
      companyId: "31"
    }
  },
  {
    $group: {
      _id: "$jobState",
      count: {
        $count: {}
      }
    }
  }
]
```

Below the code editor, there is a 'PIPELINE OUTPUT' section showing a sample of 2 documents. The output is:

```
{
  "_id": "ASSIGNED",
  "count": 1
}
{
  "_id": "NEW",
  "count": 4
}
```

Um das Ergebnis der Aggregation aufzunehmen, brauchen wir eine Datenklasse. Der Output der Pipeline in Compass zeigt die benötigten Attribute:

JobStateAggregationDTO
- id: String
- count: String
+ JobStateAggregationDTO()
+ getId(): String
+ getCount(): String

Füge diese Klasse im Ordner «model» deinem Projekt hinzu. Verwende Lombok für Getter und Konstruktor.

Nun braucht es im Job-Repository eine passende Query. Da wir mit dem Spring Boot Framework arbeiten (und nicht direkt mit dem MongoDB Treiber), erfolgt der Aufruf der Query etwas anders, als wir es in 3DM gemacht haben. Drücke den Button «Export to Language» und kopiere den Code für Node.

Export Pipeline To Language

My Pipeline

Shell

```
[
  {
    $match: {
      companyId: "31"
    }
  },
  {
    $group: {
      _id: "$jobState",
      count: {
        $count: {}
      }
    }
  }
]
```

☐ Include Import Statements

☐ Include Driver Syntax

Exported Pipeline

Node

```
[
  {
    '$match': {
      'companyId': '31'
    }
  }, {
    '$group': {
      '_id': '$jobState',
      'count': {
        '$count': {}
      }
    }
  }
]
```

Close

Entferne die eckigen Klammern am Anfang und am Ende. Entferne zudem die Zeilenumbrüche so dass beide Stages jeweils auf einer Zeile stehen.

```
{'$match': {'companyId': '31'}}
{'$group': {'_id': '$jobState', 'count': {'$count': {}}}}
```

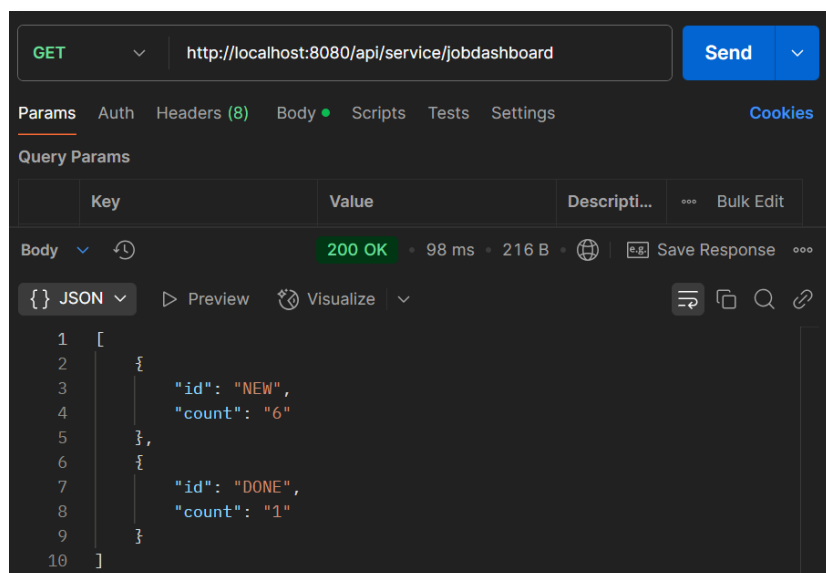
Füge die Pipeline wie unten gezeigt als Parameter bei @Aggregation im Job-Repository ein.

```
@Aggregation({
    "{$match": {'companyId': '31'}}",
    "{$group": {'_id': '$jobState', 'count': {'$count': {}}}}")
})
List<JobStateAggregationDTO> getJobStateAggregation();
```

Nun fehlt nur noch der passende Endpoint. Der folgende Code kommt den JobServiceController.

```
@GetMapping("/jobdashboard")
public List<JobStateAggregationDTO> getJobStateAggregation() {
    return jobRepository.getJobStateAggregation();
}
```

Für den Zugriff auf das jobRepository muss du dieses per @Autowired einbinden. Teste mit Postman, ob sich der Endpoint wie gewünscht verhält.



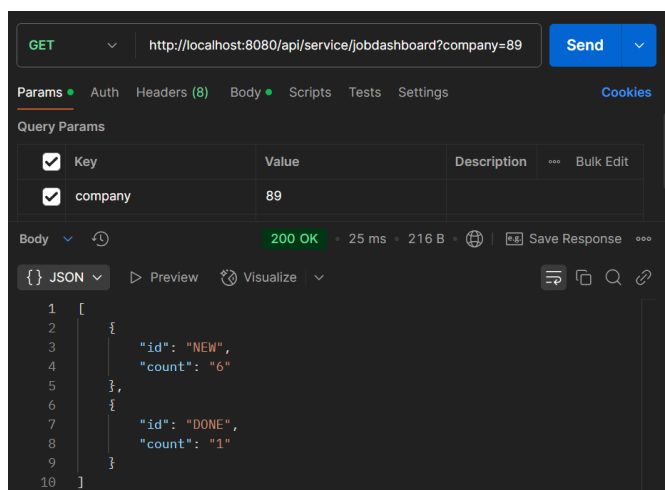
Aktuell ist die CompanyId fix auf 31 gesetzt. Diese sollt jedoch über den Request-Parameter «company» gesetzt werden können. Damit dies funktioniert, braucht es eine Anpassung bei der Query:

```
@Aggregation({
    "{$match": {'companyId': '?0'}}",
    "{$group": {'_id': '$jobState', 'count': {'$count': {}}}}")
})
List<JobStateAggregationDTO> getJobStateAggregation(String companyId);
```

Nun kann auch der Controller entsprechend angepasst werden.

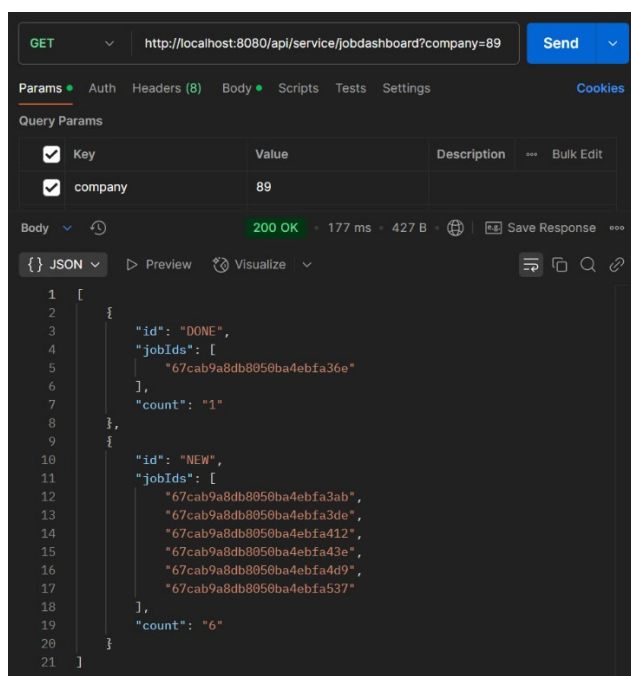
```
@GetMapping("/jobdashboard")
public List<JobStateAggregationDTO> getJobStateAggregation(@RequestParam String company) {
    return jobRepository.getJobStateAggregation(company);
}
```


Beim Request sollte nun die Company gewählt werden können.



Die Aggregation ist noch nicht ganz vollständig. Es fehlen noch die jobIds. Ergänze den Endpoint so, dass zusätzlich die JobIds ausgegeben werden. Die wichtigsten Schritte:

- In Compass Stage 2 der Pipeline so erweitern, dass zusätzlich auch die Job-IDs im Output enthalten sind. Tipp: du benötigst den Operator «push».
- Die Änderungen der Pipeline in das jobRepository übernehmen.
- Das JobStateAggregation-Model um den zusätzlichen Output ergänzen.
- Die Änderungen mit Postman testen.



Schliesse auch dieses Issue mit einer entsprechenden Commit-Message. Damit hast du alle Issues umgesetzt. Der Sprint ist geschafft. Glückwunsch! 😊

Abgabe: Prüfe, ob du alle Änderungen ins Remote-Repo gepusht hast. Gib anschliessend nochmals die URL des Repos in Moodle ab, um zu bestätigen, dass du die Übung abgeschlossen hast.